Python Best Practices

Python Best Practices

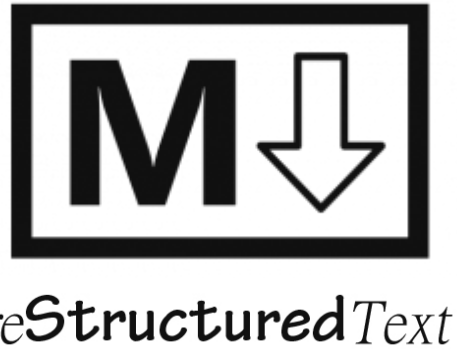
Like any other language or tool, Python has some best practices to follow before, during, and after the process of writing your code. These make the code readable and create a standard across the industry. Other developers working on the project should be able to read and understand your code. We have listed out a few of these for you to follow and write cleaner and more professional code. Do you follow any of these?

So, here come some important Python best practices which you should always keep in mind.

1. Create a Code Repository and Implement Version Control

If you have ever been on GitHub, you must have noticed that a regular project's structure looks like this:

docs/conf.py
docs/index.rst
module/__init__.py
module/core.py
tests/core.py
LICENSE
README.rst
requirements.txt
setup.py



Structure of the Python Project

When you start a new python project, you can create a code repository and implement version control. GitHub is one provider of this, but you can use any other. Let's talk about the structure of a project, but before that, I recommend you to check DataFlair's latest python project on *Speech Emotion Recognition*.

License

This is in the root directory and is where you should add a license for your project. Some licenses are:

GNU AGPLv3 **GNU GPLv3** **GNU LGPLv3** **Mozilla Public License 2.0** **Apache License 2.0**

MIT License **The Unlicense**

Structure of the Python Project

README

This is in the root directory too and is where you describe your project and what it does. You can write this in Markdown, reStructuredText, or plain text.

Module Code

This holds your actual code that may be inside a subdirectory or inside root.

requirements.txt

This is not mandatory, but if you use this, you put it in the root directory. Here, you mention the modules and dependencies of the project- the things it will not run without.

Structure of the Python Project

setup.py

This script in the root lets distutils build and distribute modules needed by the project.

Documentation

Readable documentation is essential. This is placed in the docs directory.

Tests

Most projects have tests- keep these in the tests directory.

2. Create Readable Documentation

So, next in python best practices is readable documentation. Like we just said, readable documentation is important. You may find it burdensome, but it creates clean code. For this, you can use Markdown, reStructuredText, Sphinx, or docstrings. Markdown and reStructuredText are markup languages with plain text formatting syntax to make it easier to mark up text and convert it to a format like HTML or PDF. reStructuredText lets you create in-line documentation. And Sphinx is a tool to easily create intelligent and beautiful documentation. It lets you generate Python documentation from existing reStructuredText. It also lets you export documentation in formats like HTML. Docstrings are documentation strings at the beginning of each module, class, or method.

3. Follow Style Guidelines

The PEP8 holds some great community-generated proposals. PEP stands for Python Enhancement Proposals- these are guidelines and standards for development. There are other PEPs like the PEP20, but PEP8 is the Python community bible for you if you want to properly style your code. This is to make sure all Python code looks and feels the same. One such guideline is to name classes with the CapWords convention.

- Use proper naming conventions for variables, functions, methods, and more.
- Variables, functions, methods, packages, modules: this_is_a_variable
- Classes and exceptions: CapWords
- Protected methods and internal functions: _single_leading_underscore
- Private methods: __double_leading_underscore
- Constants: CAPS_WITH_UNDERSCORES

Use 4 spaces for indentation. For more conventions, refer to PEP8.

4. Correct Broken Code Immediately

Like with the broken code theory, correct your broken code immediately. If you let it be while you work on something else, it can lead to worse problems later. This is what Microsoft does. It once had a terrible production cycle with MS Word's first version. So now, it follows a 'zero defects methodology', and always corrects bugs and defects before proceeding.

5. Use the PyPI Instead of Doing it Yourself

One of the reasons behind Python's popularity is the PyPI- this is the Python Package Index; it has more than 198,190 projects at the time of writing. This is a repository of software for Python and has thousands of Python projects. You should use code from this instead of writing it yourself- this saves time and lets you focus on the more important things. The PyPI has projects about everything, and you can install these using pip. You can also create and upload your own package here. Follow this best practice in Python, this will help you a lot.

6. The Zen of Python

Tim Peters wrote this short poem to express what values you should follow while coding in Python. You can get this by running “import this” in the IDLE:

```
>>> import this
```

The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren’t special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one– and preferably only one –obvious way to do it.

7. Use the Right Data Structures

Know the benefits and limitations of different *data structures*, and choose the right one for your code.

8. Write Readable Code

You should use line breaks and indent your code. Use naming conventions for identifiers- this makes it easier to understand the code. Use comments, and whitespaces around operators and assignments. Keep the maximum line length 79 characters. You should stay consistent and write readable code.

9. Use Virtual Environments

You should create a virtual environment for each project you create. This will avoid any library clashes, since different projects may need different versions of a certain library.

10. Write Object-Oriented Code

Python is an object-oriented language, and everything in Python is an object. You should use the object-oriented paradigm if writing code for Python. This has the advantages of data hiding and modularity. It allows reusability, modularity, polymorphism, data encapsulation, and inheritance. Also, you should write modular and non-repetitive code. Use classes and functions in your code.

11. What Not to Do

Avoid importing everything from a package- this pollutes the global namespace and can cause clashes. Don’t implement best practices from other languages. Don’t turn off error reporting during development- turn it off after it. Don’t alter sys.path, use distutils for that.

HTTP requests

HTTP (HyperText Transfer Protocol)

- HTTP stands as the foundational protocol in the web world, designed for transferring hypertext documents, such as HTML. It underpins the entire web, allowing web browsers (clients) to request web pages from web servers. Through various request methods, HTTP enables performing diverse actions, such as retrieving, sending, updating, and deleting data on the server.

URL (Uniform Resource Locator)

- A URL serves as the universal method for addressing resources on the internet, encompassing web pages, images, and files. Each URL contains a protocol for communication (e.g., HTTP or HTTPS), a domain name pointing to the web server where the resource is hosted, and a path specifying the resource's location on the server. This allows web browsers to accurately locate and retrieve resources from the internet.

Requests and Responses

- The exchange of requests and responses between a client and a server forms the basis of internet interaction. When a user requests a web page, their browser sends an HTTP request to the site's server. The server processes the request and sends back a response, which could include the requested page, an error message, or other data. This process is fundamental to network interaction, facilitating the retrieval, sending, and processing of web content.

HTTP Request Methods

- HTTP defines a set of request methods, each aimed at performing specific actions with resources. For instance, the GET method is used to retrieve a resource without altering it. POST allows for sending data to the server, like when filling out a form on a website. PUT and PATCH are used for updating resources, with PATCH allowing for partial updates, whereas DELETE is used to remove resources. These methods cater to different needs of interaction with web resources, providing a structured way to manage web data.

Status Codes

- HTTP status codes are critical in indicating the outcome of the request made to the server. They are divided into several categories: 1xx (Informational), 2xx (Success), 3xx (Redirection), 4xx (Client Error), and 5xx (Server Error). For instance, a 200 OK status code means the request was successfully processed, while a 404 Not Found indicates that the requested resource could not be found. These codes help in diagnosing issues and understanding the server's response to a given request.

Headers and Body in HTTP

- In HTTP communication, headers and body play distinct roles. Headers provide metadata about the request or response, such as content type, length, and cookies, facilitating the data exchange between client and server. The body of the request or response carries the actual data being transmitted, like form data in a POST request or the content of a web page in a response. Understanding the structure and purpose of headers and body is crucial for effective web development and API interaction.

REST (Representational State Transfer)

- REST is an architectural style for designing networked applications, relying on stateless communication and using HTTP as its underlying protocol. It emphasizes resources rather than actions, with each resource identified by a URL. RESTful applications use HTTP request methods to create, read, update, or delete resources, making web services more scalable and simple. This approach has become widely adopted for API development, thanks to its flexibility and ease of integration.

Data Formats: JSON and XML

- In web services and API communication, data is often exchanged in standardized formats, with JSON (JavaScript Object Notation) and XML (eXtensible Markup Language) being the most common. JSON is lightweight and easily readable, making it highly popular for web APIs. XML is more verbose and allows for a greater level of structure and schema definition, suited for complex data exchanges. Both formats play crucial roles in modern web development, enabling structured and efficient data interchange.

Libraries for HTTP Requests in Python

- Python offers several libraries to make HTTP requests simpler and more efficient. The requests library is renowned for its user-friendly interface, allowing for easy sending of HTTP requests and handling of responses. For lower-level interactions, http.client provides classes and methods for making HTTP and HTTPS connections. The urllib library, though more complex, offers a wide range of capabilities for URL handling, including making HTTP requests. Choosing the right library depends on the specific needs of your application, such as simplicity, control, or compatibility.

Session Objects in Python

- Session objects in Python, especially within the requests library, enable persistence across HTTP requests. Sessions allow for the reuse of certain parameters, such as cookies and headers, making subsequent requests more efficient by maintaining state information like authentication. This feature is particularly useful for applications that interact with web services requiring login credentials or custom headers across multiple requests.

Cookies in HTTP

- Cookies are essential for state management in web applications, allowing servers to send state information to the client's browser, which stores it and returns it with subsequent requests. This mechanism supports functionalities like user sessions, personalized content, and tracking user activity across sessions. Understanding how to handle cookies is crucial for both server-side logic and client-side interactions, ensuring secure and efficient state management.

Security Considerations in HTTP Requests

- Security is paramount in handling HTTP requests, especially when dealing with sensitive data. Practices such as using HTTPS for encrypted communication, validating SSL certificates, and careful management of cookies and session data are essential. Additionally, being cautious with data received from external sources and validating it before processing can protect against common vulnerabilities like injection attacks. Ensuring security in HTTP communication not only protects data integrity but also maintains user trust in your application.

Library requests

Introduction to the `requests` Library

- The requests library in Python is a user-friendly HTTP client for sending and receiving HTTP requests and responses. Unlike the built-in Python libraries like urllib, requests provides a simpler, more accessible interface for accessing web resources. It automates complex processes such as session management, supports cookies directly, handles JSON content gracefully, and deals with HTTP headers intuitively. This makes it the go-to library for interacting with RESTful APIs or any web service over the HTTP protocol.

Key Features of `requests`

- The requests library boasts several features that make it incredibly powerful yet easy to use. Key features include: simple methods for making HTTP GET and POST requests; automatic handling of URL encoding; support for form data, multipart file uploads, and headers customization; SSL certificate verification; cookies persistence across requests; and session objects to maintain state and persist settings across requests. This comprehensive functionality covers the majority of needs for modern web interaction in Python.

Getting Started with requests

- Getting started with requests is straightforward. If not already installed, you can add it to your Python environment using pip, Python's package manager, with the command: pip install requests. This single command prepares your environment to leverage the full capability of requests for making HTTP requests.

Making GET Requests

- One of the most common operations performed on the web is fetching data using GET requests. With requests, this is as simple as `response = requests.get('https://example.com/api/data')`. This line of code sends a GET request to the specified URL and stores the response from the server in the response object. You can access the content of the response using response.text for plain text responses or response.json() for JSON responses, making data extraction seamless.

```
import requests

response = requests.get('https://example.com/api/data')

print(response.text)

json_response = response.json()
print(json_response)
```

Handling Responses

- Every HTTP request sent using requests returns a response object containing the server's response. This object provides .status_code to check the HTTP status code, .headers to access response headers, and methods like .text or .json() to parse the response body. Handling different types of responses is crucial for effectively processing data received from the web.

```python
import requests

response = requests.get('https://example.com/api/data')

if response.status_code == 200:
    print("Request successful!")
else:
    print("Request failed with status code:", response.status_code)

print("Response headers:", response.headers)
print("Response content:", response.text)

try:
    json_response = response.json()
    print("JSON response:", json_response)
except ValueError:
    print("Response does not contain valid JSON.")
```

Sending POST Requests

- POST requests are essential for submitting data to web services. With requests, you can easily send POST requests with data or files. For example, requests.post(url, data={'key': 'value'}) sends a form data POST request, while requests.post(url, json={'key': 'value'}) sends a POST request with JSON data. This flexibility allows for interaction with various web services requiring data submission.

```python
import requests

url = 'https://api.example.com/submit'

form_data = {'key': 'value'}

response = requests.post(url, data=form_data)

if response.status_code == 200:
    print("Успішно надіслано дані!")
    print("Відповідь сервера:", response.text)
else:
    print("Помилка при відправці даних:", response.status_code)
```

Authentication with requests

- Secure web services often require authentication. requests simplifies this process by providing built-in support for various authentication mechanisms. For basic authentication, use requests.get(url, auth=('user', 'pass')), which seamlessly handles the authentication header. For more complex scenarios, requests supports OAuth and other authentication methods, making secure communication with web services straightforward.

```python
import requests

url = 'https://api.example.com/data'

auth_params = ('username', 'password')

response = requests.get(url, auth=auth_params)

if response.status_code == 200:
    print("Дані отримано успішно!")
    print("Вміст відповіді:", response.text)
else:
    print("Помилка при отриманні даних:", response.status_code)
```

Working with Cookies

- Cookies are crucial for maintaining session state with web services. requests allows for easy manipulation of cookies. You can send cookies to a server using the cookies parameter in your request and access server-set cookies via the response.cookies attribute. This feature is particularly useful for scripts that interact with web services requiring login sessions.

```
import requests

response = requests.get('https://api.example.com/data', cookies={'session_id': '123456'})

if response.status_code == 200:
    print("Запит успішно виконано!")
    print("Вміст відповіді:", response.text)
else:
    print("Помилка при виконанні запиту:", response.status_code)

received_cookies = response.cookies
print("Отримані cookies:", received_cookies)
```

Sessions in requests

- For prolonged interaction with a web service, maintaining a session can optimize network communication. requests offers session objects that persist cookies and headers across multiple requests. Using a session (with requests.Session() as session:), you can make several requests with shared cookies and other HTTP parameters, which is ideal for web scraping or interacting with web APIs that require authentication.

```
import requests

with requests.Session() as session:
    response1 = session.get('https://example.com/login')
    response2 = session.post('https://example.com/login', data={'username': 'user', 'password': 'pass'})
    response3 = session.get('https://example.com/dashboard')
    response4 = session.get('https://example.com/profile')
```

Error Handling and Exceptions

- Robust error handling is critical for resilient applications. requests raises exceptions for HTTP errors, connection problems, and timeouts, which can be caught and handled in your code. For instance, requests.exceptions. RequestException is a base class for most exceptions raised by requests, allowing for broad error handling, while more specific exceptions like requests.exceptions.HTTPError can be used to handle specific HTTP status codes.

```python
import requests

try:
    response = requests.get('https://example.com/api')
    response.raise_for_status()

except requests.exceptions.HTTPError as err:
    print(f"HTTP error occurred: {err}")

except requests.exceptions.ConnectionError as err:
    print(f"Error connecting to the server: {err}")

except requests.exceptions.Timeout as err:
    print(f"Timeout error: {err}")

except requests.exceptions.RequestException as err:
    print(f"An unexpected error occurred: {err}")
```

Practical Examples with requests

- Practical applications of requests range from simple data fetching to complex interactions with web APIs. Examples include web scraping, interacting with RESTful APIs for data analysis, automating interactions with web forms, and scripting for automated testing of web applications. Through these examples, we see requests as a versatile tool for all levels of web interaction.

Conclusion and Resources

- The requests library is an essential tool for Python developers working with HTTP. Its simplicity and power facilitate a wide range of web interaction tasks. For further exploration, consult the requests documentation, engage with community forums, and experiment with the library to enhance your web programming skills.
- https://requests.readthedocs.io/en/latest/

Q&A